

Session – 8: Functional Programming using reduce():-

8a: Maximum Subarray –

Program :

```
class Solution {  
    public int maxSubArray(int[] nums) {  
        int maxSum = Integer.MIN_VALUE;  
        int currentSum = 0;  
        for (int i=0;i<nums.length;i++){  
            currentSum += nums[i];  
            if(currentSum > maxSum){  
                maxSum = currentSum;  
            }  
            if(currentSum < 0){  
                currentSum = 0;  
            }  
        }  
        return maxSum;  
    }  
}
```

Output :

The screenshot shows a LeetCode interface with the following components:

- Problem List:** Shows the problem is "Accepted" with 210/210 testcases passed. A badge indicates a runtime of 1 ms, beating 99.92% of solutions.
- Runtime and Memory:** Runtime is 1 ms (Beats 99.92%), Memory is 76.89 MB (Beats 95.68%).
- Code Editor:** Displays the Java code for the Maximum Subarray problem, which uses a brute-force approach with nested loops to find the maximum sum of a contiguous subarray.
- Test Result:** Shows the test case "Accepted" with a runtime of 0 ms. The input is `nums = [-2,1,-3,4,-1,2,1,-5,4]` and the output is `6`.

8b: Find The Highest Altitude –

Program:-

```
class Solution {  
    public int largestAltitude(int[] gain) {  
        int altitude = 0;  
        int maxAltitude = 0;  
        for (int g : gain) {  
            altitude += g;  
            maxAltitude = Math.max(maxAltitude, altitude);  
        }  
        return maxAltitude;  
    }  
}
```

Output :

The screenshot displays the LeetCode submission page for the problem "Find the Highest Altitude". The submission is marked as "Accepted" with 210 / 210 testcases passed. The user "Praveen_1437" submitted it on Sep 15, 2026 at 11:12. A banner for "Unlock the Full LeetCode Experience" is visible, offering a ₹583.25/month plan. The performance metrics show a runtime of 1 ms (Beats 99.92%) and memory usage of 76.89 MB (Beats 95.68%). A bar chart shows the runtime performance relative to other submissions. The code editor shows the Java solution for the problem. The test result section shows three cases passed, with the input [-2, 1, -3, 4, -1, 2, 1, -5, 4] resulting in an output of 6.

Accepted 210 / 210 testcases passed
Praveen_1437 submitted at Sep 15, 2026 11:12

Unlock the Full LeetCode Experience
₹583.25/mo
Save over 96%
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime 1 ms | Beats 99.92%
Memory 76.89 MB | Beats 95.68%

Code | Java

```
1 class Solution {  
2     public int maxSubArray(int[] nums) {  
3         int maxSum = Integer.MIN_VALUE;  
4         int currentSum = 0;  
5         for (int i=0; i<nums.length; i++){  
6             currentSum += nums[i];  
7             if(currentSum > maxSum){  
8                 maxSum = currentSum;  
9             }  
10            if(currentSum < 0){  
11                currentSum = 0;  
12            }  
13        }  
14    }  
15 }
```

Testcase | **Test Result**

Accepted Runtime: 0 ms

Case 1 **Case 2** **Case 3**

Input

nums =
[-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output

6

Expected

6